Breaking the Kerckhoffs Geocache Cipher

A step-by-step walkthrough for the curious undergraduate

Abstract

Three geocaches hide a custom polyalphabetic cipher. The first publishes its whole table, the second hides the table but keeps two conveniences, and the third removes both conveniences and looks unbreakable. This document walks through how each part was solved, slowly enough to follow with basic linear algebra, a little statistics, and introductory algorithms. The point is not just the answer. It is to show how a handful of standard ideas—modular arithmetic, linear systems over finite fields, frequency statistics, bitmasks, and greedy algorithms—combine into a real cryptanalysis.

Contents

1 The setting: Kerckhoffs's principle

Auguste Kerckhoffs argued that a cryptosystem should stay secure even if everything about its design is public except the key. The cache series is a playful demonstration. Each part uses the same basic machine:

1. There is a table mapping the 26 plaintext letters to 26 digit strings called *codewords*.
2. The table is used to replace each letter by its codeword.
3. After each codeword is used, the whole table is rotated by a shift value. The rotation is cumulative, so the same letter can be encoded by different codewords at different positions.

Part 1 publishes the table. Part 2 hides the table but keeps two hints in the ciphertext: the first digit of a codeword says how long it is, and the last digit says how much to rotate afterwards. Part 3 removes both hints: the first digit is meaningless and the shift is a secret value attached to each codeword.

Key idea

The whole series is a lesson in *known-plaintext attacks*. A cipher can look fearsome because the key space is huge, yet collapse the moment an attacker knows a long stretch of plaintext. That is exactly what happens in part 3.

2 Part 1: the table is public

2.1 The machine

Let the public table be a list of 26 codewords, one per letter. In the actual puzzle the file alternates letters and codewords:

```
A 89411894
B 52381
C 88527391
...
Z 4502
```

Each codeword begins with its own length: `89411894` has length 8 and indeed starts with 8; `52381` has length 5 and starts with 5. This is a *prefix-free* code: no codeword is a prefix of another, because the first digit tells the decoder where the codeword ends.

After encoding a letter, the table is rotated so that every letter now maps to the codeword that used to belong to the previous letter. Concretely, if the table is a list of 26 codewords, a rotation by k moves each codeword down k slots (wrapping around). The rotation amount is the *last digit* of the codeword that was just used.

2.2 Decryption by hand

Because the first digit gives the length, decryption is a straightforward greedy scan:

1. Read the first digit; that is the codeword length.
2. Slice out the codeword.
3. Look up its plaintext letter in the current table.
4. Rotate the table by the codeword's last digit.
5. Repeat from the new position.

The decrypted message contains the final coordinates:

```
THE CACHE IS LOCATED AT NORTH THIRTY NINE DEGREES TEN POINT FOUR EIGHT TWO ...
```

What this teaches

Greedy decoding works because the code is prefix-free. The first digit is an explicit length field, so the parser never has to backtrack. This is the same idea as length-prefixed strings in binary formats.

3 Part 2: the table is hidden

3.1 What is still visible

Two things survive:

- The first digit of each codeword still says how long it is, so the ciphertext can be split into codewords by a simple scan.
- The last digit still says how far the table rotates after that codeword, so the cumulative shift before every position is known.

Let c_i be the codeword at position i, let $\mathrm{pos}(c_i)$ be its position in the *original* table, and let S_i be the cumulative rotation before position i. Then

$$\boxed{p_i \;=\; \big(\mathrm{pos}(c_i) + S_i\big) \bmod 26} \tag{1}$$

where p_i is the i-th plaintext letter as a number $0, \ldots, 25$. The shifts S_i are known from the ciphertext. The only unknown is the permutation pos.

3.2 The search space and a smarter move

There are 26! possible tables—about 4×10^{26}. Brute force is hopeless. But the problem has a very useful shape: if we guess a table, we can decrypt the whole message and ask, "does this look like English?"

That is a classic optimisation setup:

- **State:** a permutation of the 26 table positions.
- **Score:** how English-like the decrypted text is.
- **Move:** swap two entries of the permutation.

3.3 Scoring English

A *quadgram* is a sequence of four letters, e.g. `THEM`. The repository includes counts of quadgrams from a large English corpus. If a candidate decryption contains quadgram q, we add $\log P(q)$ to its score. For a text of length n the total score is

$$\mathrm{score}(p_1 \ldots p_n) \;=\; \sum_{i=1}^{n-3} \log P(p_i p_{i+1} p_{i+2} p_{i+3}). \tag{2}$$

Logs are used for two reasons: they turn a product of probabilities (which would underflow) into a sum, and the sum is cheap to update when only a few letters change.

What this teaches

Language models are just probability distributions over short letter sequences. Turning a product into a sum of logs is a standard numerical trick, and it makes incremental rescoring possible.

3.4 Simulated annealing

Simulated annealing is a randomised hill-climber. At each step we propose a swap. If the score improves we accept it. If it gets worse we accept it with probability

$$\exp\!\left(\frac{\Delta}{T}\right), \tag{3}$$

where Δ is the score change and T is a *temperature* that starts high (explore) and slowly cools (exploit). This lets the search escape local optima early on and settle into a good solution later.

The implementation is careful about speed. When we swap the entries for symbols a and b, only positions whose codeword is a or b change their plaintext letter, and only quadgrams overlapping

those positions change. A precomputed list of positions per symbol (a hash map from symbol to positions) makes each move cost roughly the number of affected positions rather than the length of the text.

What this teaches

The algorithmic idea is *incremental evaluation*: never recompute more than the move actually changed. This is the same reason balanced trees and hash maps matter in algorithmic problem solving.

The annealing recovers the table and decrypts the message. It also reveals the coordinate instruction:

```
TO SOLVE SWAP AND DOUBLE THE LAST PARTS OF THE COORDS
```

Applying it to the posted coordinates $N\,39°\,10.227$, $W\,076°\,51.187$ means swapping the decimal minutes ($227/187 \to 187/227$) and doubling them ($187/227 \to 374/454$), giving the confirmed final coordinate $N\,39°\,10.374$, $W\,076°\,51.454$. Applying the same recipe to part 1's numbers is a trap — the checker rejects it.

4 Part 3: both conveniences removed

Part 3 changes two rules:

- the first digit no longer gives the codeword length;
- the last digit no longer gives the shift: every codeword has a secret shift value in $0, \ldots, 25$.

Without the length prefix, the ciphertext cannot even be split into codewords without knowing the dictionary. Without the shift digits, the cumulative shift is part of the key. This section is the heart of the document.

4.1 Recovering the codeword dictionary

The plaintext is a concatenation of codewords, but we do not know where the boundaries are. However, the code is still prefix-free, and the codewords repeat often: a codeword encodes a fixed plaintext letter (up to the table rotation), so a common letter's codeword appears many times.

A statistical argument. Suppose the dictionary were replaced by random digit strings and the text were random. The expected number of occurrences of a fixed length-L string in N digits is about $N/10^L$. For $N = 3377$ and $L = 9$ that is 3.4×10^{-6}—you would essentially never see a 9-digit string repeat. Yet the recovered length-9 codewords appear 18–26 times each. That is overwhelming evidence that they are real codewords rather than coincidences.

The recovery procedure.

1. Count occurrences of every substring of length $2, \ldots, 9$ with a hash map. This is $O(9N)$ work.
2. Keep the most frequent maximal strings; these are codeword candidates.

3. Greedily segment the whole ciphertext with the candidates, longest first. A prefix-free set makes the segmentation unambiguous.

4. Verify that the set tiles the entire ciphertext with no leftovers and that no candidate is a prefix of another.

The result is 26 codewords, e.g.

```
243255912 222146337 946294413 80920926 63388773
37638005 6103003 0273351 2167948 8621836
...
860 781 418 0648 30 93
```

4.2 The key and the model

Now the decryption rule is

$$p_i = \big(\tau[c_i] + S_i\big) \bmod 26, \tag{4}$$
$$S_{i+1} = \big(S_i + \omega[c_i]\big) \bmod 26, \tag{5}$$

where

- c_i is the (now known) codeword index at position i;
- τ is a permutation of $0, \ldots, 25$ (the hidden table order);
- $\omega[c]$ is the secret rotation after codeword c.

There are 26 values of τ and 26 values of ω: 52 unknowns. The naive key space is $26! \cdot 26^{26}$, which is astronomically large.

4.3 The linear structure that makes it solvable

The secret shifts look intimidating, but the cipher is *linear*. Subtract the rule at consecutive positions:

$$\begin{aligned} p_{i+1} - p_i &= \big(\tau[c_{i+1}] + S_{i+1}\big) - \big(\tau[c_i] + S_i\big) \\ &= \tau[c_{i+1}] - \tau[c_i] + \omega[c_i] \pmod{26}. \end{aligned} \tag{6}$$

Everything on the right is a sum of key values with coefficients $0, \pm 1$. If we know a stretch of plaintext—a *crib*—then equation (**??**) gives one linear equation per adjacent pair of letters, plus one absolute equation for the first letter. That is a linear system in the 52 unknowns over the ring $\mathbb{Z}/26\mathbb{Z}$.

Key idea

A crib of roughly 100 letters gives more than 52 equations. The system is overdetermined and usually has a unique solution for the key, even though the key space looked like $26! \cdot 26^{26}$. The apparent hardness was the missing plaintext, not the key size.

4.4 Solving the linear system

The modulus factors as $26 = 2 \cdot 13$, and both 2 and 13 are prime. By the Chinese remainder theorem, solving modulo 26 is the same as solving modulo 2 and modulo 13 separately and combining the answers. Over a prime field, ordinary Gaussian elimination works: we can divide by any nonzero coefficient.

1. Build the equations from the crib.
2. Perform Gaussian elimination modulo 2 and modulo 13.
3. If either system is inconsistent, the crib is wrong (or misplaced).
4. Otherwise read off free variables, enumerate their small number of possibilities, and keep only keys whose τ is a permutation.
5. Decrypt and score with the quadgram model.

What this teaches

Finite fields, Gaussian elimination, and the Chinese remainder theorem are not just abstract algebra. They are exactly the tools that turn a huge discrete search into a small linear-algebra computation.

4.5 Finding the crib: hints and a two-block search

Part 3 has no public table and no checker, so the crib is the whole game. The hint block eventually supplied:

> [Puzzle 4] The two phrases `North Thirty Nine Degrees Ten Point` and `West Seventy Six Degrees Fifty One Point` both appear somewhere in the plaintext. No other assumptions about the plaintext need be made.

The earlier hint ("only the seconds differ") only guarantees that the final coordinate shares its degrees and minutes with the posted one. Hint 4 is what gives *exact* plaintext.

Write the two phrases as

$$A = \texttt{NORTHTHIRTYNINEDEGREESTENPOINT} \qquad (30 \text{ letters}),$$
$$B = \texttt{WESTSEVENTYSIXDEGREESFIFTYONEPOINT} \qquad (34 \text{ letters}).$$

Together A and B are 64 known letters, more than the 52 unknowns. The catch is that we do not know their offsets, and they are *not* adjacent and not in the same sentence position as in part 1.

Searching all offset pairs. There are about 550 offsets for A and 545 for B, so roughly 3×10^5 pairs. For each pair we build the combined linear system over $\mathbb{Z}/2\mathbb{Z}$ and test consistency.

Why check modulo 2 first? Because each equation can be packed into a Python integer: 52 bits for the coefficients and one bit for the right-hand side. A 53-bit integer fits in a machine word, and elimination on such rows is fast. The entire $\sim 3 \times 10^5$-pair search runs in about one second in ordinary Python.

```python
MASK = (1 << 52) - 1 # 52 coefficient bits; bit 52 is the RHS

def consistent(a_pivots, b_rows):
    local = [0] * 53
    for row in b_rows:
        r = row
        while True:
            coeff = r & MASK
            if coeff == 0:
                if (r >> 52) & 1:
                    return False # reduced to 0 = 1
                break
            c = coeff.bit_length() - 1 # highest set coefficient
            p = a_pivots[c] or local[c]
            if p:
                r ^= p # GF(2) addition is XOR
            else:
                local[c] = r
                break
    return True
```

What this teaches

Choosing the right representation is an algorithmic superpower. "Vectors over GF(2)" become bitmasks, XOR becomes vector addition, and Gaussian elimination becomes a tight loop over machine words.

The search finds one pair that survives modulo 2 *and* modulo 13: A at symbol 46, B at symbol 114. The system still has 6 free dimensions modulo 2 and 6 modulo 13, so the key is not yet unique.

Reading the gap. Decrypting with any convenient representative of the solution space (free variables set to zero) exposes the text between the two phrases. It begins

`FIVEHUNDREDTHIRTEENINTHEOTHERDIRECTION`

That is the latitude seconds `FIVEHUNDREDTHIRTEEN` $= 513$ followed by the connector `INTHEOTHERDIRECTION`. Adding this 38-letter gap as a final crib pins the key uniquely. The recovered key is

$$\tau = [9, 16, 15, 11, 2, 12, 23, 19, 10, 18, 5, 1, 0, 17, 25, 3, 24, 7, 13, 4, 22, 20, 8, 6, 21, 14],$$
$$\omega = [17, 13, 5, 21, 21, 17, 2, 5, 17, 10, 4, 14, 0, 1, 11, 18, 14, 24, 1, 5, 1, 24, 11, 5, 18, 19].$$

4.6 The plaintext and the final answer

The decrypted message reads (spaced out for readability):

> WOW IM IMPRESSED. I THINK THIS PUZZLE IS PRETTY HARD. GO TO NORTH THIRTY NINE DEGREES TEN POINT FIVE HUNDRED THIRTEEN IN THE OTHER DIRECTION WEST SEVENTY SIX DEGREES FIFTY ONE POINT SIX EIGHT TWO. IM REALLY CURIOUS IF YOU CAN FIND A SOLUTION TO THIS

GENERALIZATION THAT DOESNT RELY ON CRIBS. THE ONLY SOLUTION I FOUND USED THE WORD THE AS THE MOST COMMON TRIGRAM STARTING WITH EACH CODEWORD. EVEN THEN IT TOOK MILLIONS OF LETTERS, WASNT CONSISTENT, AND STILL NEEDED A LARGE EXHAUST. I REALLY THOUGHT I COULD USE THE FACT THAT THERE IS A STATISTICAL DIFFERENCE IN HOW FAR AWAY THE SECOND LETTER IS CIRCULARLY FROM THE FIRST IN ALL BIGRAMS REGARDLESS OF FIRST LETTER, BUT IT DIDNT WORK. CACHE IS MAYBE KNEE HIGH IN TREE.

The coordinate sentence gives

$$\boxed{N\,39^\circ\,10.513'\quad W\,076^\circ\,51.682'}$$

and the last sentence tells the finder where to look: knee high in a tree. The posted and final coordinates differ only in the seconds, exactly as the "only the seconds differ" hint promised.

5 What was tried and why it failed

It is worth recording the dead ends, because they show what makes the puzzle genuinely hard and how the successful method differs.

5.1 Coordinate wording brute force

The earliest attempts assumed the part 3 plaintext reused part 1's coordinate sentence verbatim, with only the numbers changed. We generated many wordings of "THE CACHE IS LOCATED AT NORTH ..." with plausible connectors ("YOULL ALSO NEED", "AND WEST", etc.), number styles (digit-by-digit vs. number words), and coordinate candidates, and slid each one over every offset.

Why it failed. The hypothesis was too specific. Part 3 replaces the "THE CACHE IS LOCATED AT" prefix, uses "IN THE OTHER DIRECTION" instead of "YOULL ALSO NEED", and the two coordinate phrases are not adjacent. Every one of those differences makes the crib text wrong, and a wrong crib is rejected by the linear system. No amount of sliding fixes a wrong *word*.

5.2 Frequency and difference statistics without a crib

Several crib-free attacks were tried:

- **Letter-frequency annealing.** Optimise the key so that the decrypted letter frequencies match English.
- **Bigram-frequency annealing.** Same idea with two-letter frequencies.
- **Letter-difference distribution.** In any English bigram xy, the circular difference $y - x \bmod 26$ has its own distribution. Group the 676 codeword pairs by this difference and optimise the key to match the expected distribution.
- **Statistical recovery of the pair-difference table.** Because $p_{i+1} - p_i$ depends only on the codeword pair (c_i, c_{i+1}), the 26×26 table of pair differences is a Latin square. One can try to recover it from the observed pair counts using an assignment (Hungarian) algorithm. With only about 580 pairs and counts of 1–5 per cell, however, the counts are far too noisy to pin the table down.

- **Letter-difference trigrams.** Build the distribution of three consecutive differences from a quadgram corpus and anneal against it.

Why they failed. All of these objectives throw away the order of the text. They are low-dimensional summaries, so many wrong keys produce statistics that look plausible. The resulting optimisation landscape is full of local optima, and randomised search gets stuck. On synthetic data with a known key, none of these methods recovered the key. The author's own note in the plaintext says the same thing happened to him: even using `THE` as a crib took millions of letters, was inconsistent, and still needed a large exhaustive search. The statistical signal is real but far too weak on a 579-letter message.

5.3 Short cribs and underdetermined systems

Early attempts also tested short phrases from the hints (for example `ONLYTHESECONDSDIFFER`) at every offset. Each phrase is shorter than the 52 unknowns, so the linear system has more unknowns than equations: it is *underdetermined*, and almost every placement is consistent. The consistency test therefore filters nothing, and the solver can only report that the solution space is too large.

The lesson is a dimension count. For a unique key you need at least as many independent equations as unknowns, and in practice a comfortable margin. A 30-letter phrase gives 30 equations against 52 unknowns; two 30–40 letter phrases together give over 60 and become useful.

5.4 Why the crib route works so much better

The crib does not summarise the text; it gives actual letters. Each adjacent pair is a hard linear constraint. With enough constraints the key is forced, and linear algebra finds it exactly. Cribs turn a statistical search into a deterministic one.

What this teaches

When a system is linear, one long known plaintext beats any amount of statistical cleverness. Known-plaintext attacks are powerful precisely because they use exact information instead of aggregate information.

6 Putting it all together

The table below maps each stage of the solve to the standard topics it uses.

Step	Concept
Part 1 decryption	Prefix-free codes, greedy parsing, modular rotation
Part 2 table recovery	Permutations, simulated annealing, n-gram language models, incremental scoring
Dictionary recovery	Hash maps, substring counting, greedy segmentation, prefix trees, birthday-style statistical reasoning
Key model	Modular arithmetic, linear equations over a ring
Solving the key	Gaussian elimination, finite fields GF(2) and GF(13), Chinese remainder theorem, free variables
Offset search	Bitmasks, XOR as vector addition, bit parallelism, complexity analysis
Gap reveal	Partial decryption, solution-space representatives
Final answer	Coordinate arithmetic, the actual cache hunt

Key idea

The puzzle is a compressed tour of applied mathematics and computer science. It starts as a data-structures exercise (prefix codes and hash maps), becomes a statistics-and-optimisation exercise (language models and annealing), then turns into linear algebra over finite fields, and finishes with a bit-level search optimisation. None of the individual pieces is exotic. The difficulty is seeing how they fit together—and, in part 3, finding the one crib that lets the linear structure do its work.

Further reading

- Kerckhoffs's principle (Wikipedia) for the security philosophy.
- Practical Cryptography's quadgram page for the language model used here.
- Any introductory linear algebra text for Gaussian elimination; any abstract algebra text for finite fields and the Chinese remainder theorem.
- The repository files `src/part1.py`, `src/part2.py`, and `src/part3.py` for the actual code, and `POSTMORTEM.md` for the short technical summary.